FASTEIG Algorithms Guide

Based on the FASTEIG source distribution

Repository version 1.0.0
Guide revised August 10, 2026

Abstract

This guide describes the eigensolvers available in FASTEIG, the numerical ideas behind them, their intended use, and the way each method is realized by the current CPU and CUDA executor architecture. The descriptions and pseudocode are implementation-aware: they document the algorithms that the repository runs today, including important limitations, rather than only their ideal textbook forms. Build, input, and command-line instructions remain in the separate FASTEIG User's Guide.

Contents

1 Scope and notation **3**
1.1 Target-selection codes . . . 3
1.2 Residuals and stopping tests . . . 4

2 Choosing a solver **4**

3 Shared numerical building blocks **5**
3.1 Executor and linear-operator model . . . 5
3.2 Krylov subspaces and matrix-free application . . . 6
3.3 Orthogonalization and block QR . . . 6
3.4 Rayleigh–Ritz extraction . . . 7
3.5 Shift-and-invert . . . 7

4 Lanczos **8**
4.1 Purpose and mathematical idea . . . 8
4.2 Implementation-faithful pseudocode . . . 8
4.3 Cost, storage, and parameters . . . 8

5 Block Lanczos **9**
5.1 Purpose and mathematical idea . . . 9
5.2 Implementation-faithful pseudocode . . . 9
5.3 Cost and cautions . . . 9

6 Krylov–Schur **10**
6.1 Purpose and mathematical idea . . . 10
6.2 Standard-problem pseudocode . . . 10
6.3 Generalized Krylov–Schur . . . 10

7 Block Krylov–Schur **11**

8 Davidson **11**

9 LOBPCG **12**

10 Chebyshev-filtered subspace iteration **13**
10.1 Filter construction 13
10.2 Implementation-faithful pseudocode 13

11 SPPC **14**

12 RMM-DIIS **14**

13 Parameter guide **15**

14 Numerical and performance guidance **16**
14.1 Accuracy checks 16
14.2 Memory and restart tradeoffs 16
14.3 CPU and GPU expectations 16

15 Implementation map **16**

16 References **17**

1 Scope and notation

FASTEIG targets a few eigenpairs of a large, sparse, real symmetric matrix. The standard problem is

$$Ax_i = \lambda_i x_i, \qquad A = A^T \in \mathbb{R}^{n\times n}, \tag{1}$$

and the generalized problem is

$$Ax_i = \lambda_i B x_i, \qquad A = A^T, \quad B = B^T \succ 0. \tag{2}$$

Only Krylov–Schur currently supports Equation (2), and that path is host-only. The other solvers discussed here operate on Equation (1).

The following notation is used throughout:

- p is the requested number of eigenpairs (the input option `nev`);
- m is the current scalar subspace dimension;
- b is a block width;
- $Q_m = [q_1, \ldots, q_m]$ is an orthonormal basis;
- $T_m = Q_m^T A Q_m$ is the projected matrix; and
- (θ_j, y_j) is a Ritz pair, where $y_j = Q_m c_j$ and $T_m c_j = \theta_j c_j$.

1.1 Target-selection codes

The input option `which` describes how projected eigenvalues are ordered. The complete target set used by the Lanczos-family implementations is:

Code	Meaning	Practical interpretation
`SR`	Smallest real/algebraic	The low end of the spectrum.
`LR`	Largest real/algebraic	The high end of the spectrum.
`SM`	Smallest magnitude	Values closest to zero; usually difficult without a transformation.
`LM`	Largest magnitude	Values farthest from zero.
`BE`	Both ends	Alternates selections from the high and low ends.
`SAI`	Shift-and-invert	Values nearest the user-supplied `shift`.

Lanczos and standard Krylov–Schur implement all six codes. Block Lanczos and Block Krylov–Schur implement `SR`, `LR`, `SM`, `LM`, and `BE`. Davidson, LOBPCG, and Chebyshev are currently end-of-spectrum implementations: `LR` and `LM` are handled by applying the method to $-A$ and negating the reported eigenvalues; other non-high-end selections are treated as low-end computations rather than as general interior targeting.

1.2 Residuals and stopping tests

The final standard-problem residual printed by the driver is

$$\rho_i = \frac{\|Ay_i - \theta_i y_i\|_2}{(1 + |\theta_i|)\,\|y_i\|_2}. \tag{3}$$

For a generalized problem it is

$$\rho_i^{(B)} = \frac{\|Ay_i - \theta_i B y_i\|_2}{(1 + |\theta_i|)\,\|By_i\|_2}. \tag{4}$$

The extra 1 keeps the scaling meaningful for eigenvalues near zero.

Intermediate progress residuals can differ from the final check. In the Lanczos family they are inexpensive estimates obtained from the coupling to the next basis vector. Other solvers may use explicit residuals or their own historical normalization. The final table is therefore the authoritative cross-solver accuracy check.

2 Choosing a solver

Table 1 summarizes the current implementation. "Native executor path" means that the large basis vectors and core recurrence can remain in executor memory. "Hybrid" means that the matrix application uses a `LinearOperator`, but substantial dense state is still stored and updated with host Eigen objects.

Table 1: Solver capabilities in the current repository.

Solver	Best use	Current execution model	Important limits
Lanczos	A few extremal eigenpairs; simple, dependable baseline; interior pairs with SAI.	Native CPU/CUDA executor path.	Unrestarted; basis storage grows as $O(nm)$.
Block Lanczos	Clusters, multiple eigenvalues, or several pairs at once.	Native CPU/CUDA path for large basis state; small block coefficients are host-visible.	Unrestarted; no SAI or generalized problem.
Krylov–Schur	Bounded-memory extremal solve; repeated restart cycles; SAI.	Standard problem has a native executor path. Generalized problem is host-only.	Standard subspace dimension is currently fixed internally; generalized SAI is unsupported.
Block Krylov–Schur	Clustered eigenvalues with restarted block expansion.	Executor-aware but presently hybrid.	Block width is `nev`; no SAI or generalized problem.
Davidson	Low or high extremal pairs when useful correction/preconditioning information is available.	Executor-aware matrix application; host dense state.	Current correction is the raw residual; no active preconditioner.
LOBPCG	Several extremal pairs with a compact three-block trial space.	Executor-aware matrix application; host dense state.	Preconditioner hook is not active in the current solver.

Solver	Best use	Current execution model	Important limits
Chebyshev	Repeatedly improving an approximate low- or high-end invariant subspace.	Executor-aware matrix application; host filtering and dense state.	Filter degree and spectral bounds strongly affect behavior.
SPPC	Hierarchical Hamiltonians with a useful leading principal problem.	Host-oriented through a legacy-operations adapter.	Specialized partition; requires `maxdim`=split size.
RMM-DIIS	Refining already reasonable eigenvector estimates.	Host-oriented through a legacy-operations adapter.	Default correction solve is a placeholder; not a robust global eigensolver by itself.

As a practical starting point:

1. Use Lanczos for a first correctness run and for CUDA SAI.
2. Use Krylov–Schur when the unrestarted Lanczos basis would be too large.
3. Use a block method for multiplicities or tightly clustered eigenvalues.
4. Use LOBPCG or Davidson when a meaningful preconditioner is available; note that the current built-in implementations do not yet activate one.
5. Use Chebyshev when a good starting subspace is available or a sequence of related eigenproblems is being solved.
6. Use SPPC only when the matrix has the hierarchical structure assumed by that method, and use RMM-DIIS as a refinement stage.

3 Shared numerical building blocks

3.1 Executor and linear-operator model

The solver-level algorithm is separated from storage and kernel choice by the `core::Executor` interface. An executor creates `Vector`, `MultiVector`, and `CsrMatrix` objects in the memory space associated with that executor. A `LinearOperator` exposes `apply`, `apply_column`, and `apply_block`; the solver does not need to know whether the operation is an Eigen sparse product, a cuSPARSE kernel, or a sparse direct solve.

For the CPU executor, an explicit matrix is reached through the existing `operations` interface and `LegacyOperationsOperator`. For the CUDA executor, stored matrices become CUDA CSR operators and ordinary products use cuSPARSE. CUDA vectors and multivectors use cuBLAS for vector and dense block operations. Dense projected eigensolves in the CUDA Lanczos workspace use cuSOLVER. CUDA SAI uses cuDSS.

This design follows the same broad separation of concerns as executor-based libraries such as Ginkgo [13]: the executor chooses placement and backend behavior, while solver pseudocode is expressed in terms of abstract objects. It does not imply that every solver is already fully device-resident; Table 2 records the present boundary.

Table 2: Current data-residency boundary.

Path	Executor-resident state	Host-resident or host-visible state
Lanczos	Basis, residual, operator result, projected workspace, Ritz-vector multiplication.	Final public Eigen vectors; scalar convergence data.
Block Lanczos	Basis blocks, operator/residual blocks, projected workspace.	Small QR and projection blocks; final public Eigen vectors.
Standard Krylov–Schur	Basis, recurrence vectors, projected workspace and standard restart operations.	Progress data and final public results.
Generalized Krylov–Schur	None of the large Krylov state is executor-backed.	Q, BQ, projected matrix, and B solves.
Block Krylov–Schur	Operator application can use the executor.	Cycle basis, projected matrix, restart, and eigensolve.
Davidson, LOBPCG, Chebyshev	Operator application can use the executor.	Dense trial spaces, orthogonalization, projection, filtering state.
SPPC, RMM-DIIS	Operator is adapted to host callbacks.	Algorithm state and correction equations.

The example GPU executable currently selects the CUDA executor automatically only for `Lanczos` and `BlockLanczos`. The standard Krylov–Schur library implementation is executor-aware, but the example driver does not yet dispatch that solver to CUDA.

3.2 Krylov subspaces and matrix-free application

Given a starting vector v, the order-m Krylov subspace is

$$\mathcal{K}_m(A, v) = \operatorname{span}\{v, Av, A^2v, \ldots, A^{m-1}v\}. \tag{5}$$

FASTEIG never requires the solver algorithm to expose an explicit A. The selected `LinearOperator` supplies the action of A, $-A$, $(A - \sigma I)^{-1}$, or a user-provided matrix-free operation. This is why the same recurrence can run with CPU CSR, CUDA CSR, or shift-invert solves.

The classes `core::KrylovSubspace` and `core::BlockKrylovSubspace` collect the shared basis-building operations: normalization, operator application, orthogonalization, block QR, and basis rotation. Solver source files still call those steps explicitly so their main loops read like the corresponding numerical algorithms.

3.3 Orthogonalization and block QR

Scalar Lanczos uses two-pass projection against the active basis. If $z = Aq_j$, the implementation computes

$$h = Q_j^T z, \qquad f = z - Q_j h, \tag{6}$$

$$s = Q_j^T f, \qquad h \leftarrow h + s, \qquad f \leftarrow f - Q_j s. \tag{7}$$

This full reorthogonalization is more expensive than the ideal three-term Lanczos recurrence, but it protects the basis from loss of orthogonality and duplicate Ritz values.

Block methods first use two-pass block Gram–Schmidt against all active basis columns. The residual block F is then factored as

$$F = QR. \tag{8}$$

The current Block Lanczos and Block Krylov–Schur paths explicitly request one-pass Cholesky QR:

$$R^T R = F^T F, \qquad Q = F R^{-1}. \tag{9}$$

It maps well to BLAS-3 kernels and is fast for well-conditioned tall-skinny blocks. The core layer also implements Householder QR, Cholesky QR2, Gram–Schmidt QR, and SVQB, on CPU and (where supported) CUDA. Cholesky QR2 repeats Cholesky QR and is more stable for less well-conditioned blocks, at roughly twice the Gram-matrix and triangular-solve work [12]. The current default remains one-pass Cholesky QR; a failed Cholesky factorization is reported rather than silently changing algorithms.

3.4 Rayleigh–Ritz extraction

For an orthonormal trial basis Q, Rayleigh–Ritz forms

$$T = Q^T A Q, \tag{10}$$

solves the small dense problem $Tc_j = \theta_j c_j$, and returns $y_j = Qc_j$. The residual is orthogonal to the trial space:

$$Q^T(Ay_j - \theta_j y_j) = 0. \tag{11}$$

This extraction appears in every subspace solver in FASTEIG. The Lanczos-family projected problem and selection helpers live in `core/RitzExtraction`; the older host-oriented solvers use the `fasteig_solver::RayleighRitz` wrappers.

For scalar Lanczos, the three-term relation gives an inexpensive residual estimate

$$\|Ay_j - \theta_j y_j\|_2 = |\beta_{m+1} e_m^T c_j|. \tag{12}$$

Block Lanczos uses the corresponding norm of the last block coupling applied to the last rows of c_j.

3.5 Shift-and-invert

For a target σ, SAI replaces A by

$$S = (A - \sigma I)^{-1}. \tag{13}$$

If $Ax = \lambda x$, then

$$Sx = \mu x, \qquad \mu = \frac{1}{\lambda - \sigma}, \qquad \lambda = \sigma + \frac{1}{\mu}. \tag{14}$$

Eigenvalues nearest σ become the largest-magnitude eigenvalues of S; therefore `SAI` orders the transformed Ritz values as `LM`, then maps them back to the original scale. Final residuals are recomputed with the original A, not trusted solely from the transformed projected problem.

CPU SAI factorizes $A - \sigma I$ once with Eigen SimplicialLDLT or MKL PardisoLDLT, depending on the build. CUDA SAI constructs a `CudaShiftInvertOperator`, analyzes and factorizes once with cuDSS, and uses a cuDSS solve for every operator application. The current CUDA path uses the conservative general/full CSR descriptor. SAI is available in Lanczos and standard Krylov–Schur, not in the block methods or the generalized problem.

4 Lanczos

4.1 Purpose and mathematical idea

Lanczos is the simplest general-purpose solver in FASTEIG and the best baseline for a few extremal eigenpairs. Applied to a symmetric operator, it constructs Q_m and a symmetric tridiagonal T_m such that

$$AQ_m = Q_m T_m + \beta_{m+1} q_{m+1} e_m^T. \tag{15}$$

The eigenpairs of T_m approximate those of A [1, 11].

The textbook method uses a three-term recurrence. The FASTEIG implementation instead reorthogonalizes each new operator result against the entire active basis. That choice improves numerical robustness but changes the orthogonalization cost from a short recurrence to a growing dense operation.

4.2 Implementation-faithful pseudocode

```
Input: operator op, initial vector x, nev p, maximum dimension m_max
Allocate executor vectors x, z and basis Q(:, 1:m_max)
Allocate executor-specific Lanczos projected workspace

for j = 1, ..., m_max
    beta_j = ||x||_2
    if beta_j is zero, generate and orthogonalize a restart vector
    q_j = x / beta_j

    z = op(q_j)
    [alpha_j, x] = two_pass_orthogonalize(Q(:,1:j), z)
    beta_next = ||x||_2
    record alpha_j, beta_j, beta_next in projected workspace

    when a progress/convergence snapshot is due
        diagonalize T_j and select the requested Ritz pairs
        estimate residuals from beta_next and the last Ritz-vector row
        stop when every requested estimate is below tol
    end
end

Form Y = Q(:,1:m) * C_selected in executor memory
If SAI, map mu to lambda = shift + 1/mu
Copy final public eigenvectors to the host and recompute final residuals
```

When `print_every` is positive, it determines the snapshot cadence. If printing is disabled, convergence is checked every five basis expansions and at the final allowed iteration. Thus the reported iteration count may differ by a few steps between numerically equivalent CPU and GPU runs because their first snapshot satisfying the tolerance can differ slightly.

4.3 Cost, storage, and parameters

With m iterations and $\mathrm{nnz}(A)$ stored nonzeros, the sparse products cost roughly $O(m\,\mathrm{nnz}(A))$. Full reorthogonalization costs $O(nm^2)$ in total and the basis requires $O(nm)$ storage. The projected eigensolves are only $m \times m$, but doing them frequently can become visible.

The principal controls are `nev`, `maxiter`, `which`, `tol`, `print_every`, `seed`, and, for `SAI`, `shift`. Here `maxiter` is both the iteration limit and the maximum basis dimension. Lanczos is unrestarted, so use Krylov–Schur if this storage is unacceptable.

5 Block Lanczos

5.1 Purpose and mathematical idea

Block Lanczos starts from a block $X_0 \in \mathbb{R}^{n\times b}$ and expands

$$\operatorname{span}\{X_0, AX_0, A^2X_0, \ldots\}. \tag{16}$$

The projected matrix is block tridiagonal. A block can capture several directions in a clustered or multiple eigenspace at once, and the dense block operations generally use modern hardware more efficiently than scalar dot products [2].

5.2 Implementation-faithful pseudocode

```
Input: operator op, initial block X0, block size b, max_blocks
Q_0 = cholqr(X0)

for k = 0, ..., max_blocks-1
    Z = op(Q_k)

    H = Q_active^T Z
    F = Z - Q_active H
    S = Q_active^T F               # reorthogonalization pass
    H = H + S
    F = F - Q_active S

    [Q_{k+1}, B_{k+1}] = cholqr(F)
    A_k = the rows of H associated with Q_k
    record A_k and B_{k+1} in the block projected matrix

    diagonalize the active block projected matrix
    estimate Ritz residuals from B_{k+1} and the final coefficient block
    stop when all requested estimates are below tol
end

Form selected Ritz vectors from all active basis blocks
Recompute final residuals with the original operator
```

The input `blksize` sets b, while `maxiter` counts block expansions rather than scalar matrix-vector products. The largest allocated basis has approximately $(\texttt{maxiter}+1)b$ columns, and the projected Ritz space after k completed blocks has dimension kb.

5.3 Cost and cautions

Each iteration applies the operator to b vectors, so the nominal sparse work is b SpMVs. Orthogonalization is block Gram–Schmidt followed by Cholesky QR and is expressed mainly with matrix-matrix kernels. Storage remains unrestarted and scales as $O(nbm_{\text{block}})$. A block size at least as large as the expected multiplicity is often useful, but an unnecessarily large block increases every sparse product, QR, and projected eigensolve.

Implementation note. The current implementation uses Cholesky QR on both CPU and CUDA. Cholesky QR is fast but assumes the residual block is sufficiently well conditioned. A rank-deficient block causes the factorization to fail; adaptive shrinking is not yet implemented. Block Lanczos does not currently support SAI or generalized problems.

6 Krylov–Schur

6.1 Purpose and mathematical idea

Krylov–Schur is a restarted Krylov method. It expands a basis to a fixed dimension, computes a Schur/Ritz decomposition of the projected problem, keeps the wanted directions, and expands again. It is closely related to thick restart Lanczos for symmetric problems, while making the restart and deflation steps explicit and flexible [3].

The key advantage over unrestarted Lanczos is bounded large-vector storage. The tradeoff is that discarded spectral information may need to be rebuilt in later cycles.

6.2 Standard-problem pseudocode

```
Choose k = min(max(2*nev, 20), n)
Create executor-resident basis Q(:,1:k), residual x, and projection workspace
Expand a Lanczos basis from column 1 through k

for restart_cycle = 1, ..., maxiter
    diagonalize the k-by-k projected matrix
    order all Ritz pairs according to which
    count converged requested pairs using projected residual estimates

    if all requested pairs converged or this is the last cycle
        form final Ritz vectors and return
    end

    retained = nev + min(number_converged, (k-nev)/2)
    rotate Q to the retained wanted Ritz vectors
    seed the compact restart projection with Ritz values and residual coupling
    expand the basis from retained+1 back to k
end
```

In this implementation `maxiter` counts restart cycles, not individual SpMVs. The standard-problem subspace dimension is presently the internal value $\min(\max(2p, 20), n)$; `maxdim` does not override it. The projected convergence test compares the absolute estimate with `tol` $|\theta|$. The final driver residual still uses Equation (3).

The standard path supports the same target codes and SAI transformation as Lanczos. Its large basis and projected workspace are executor-aware. In a CUDA-enabled library call they can remain on the device across restart cycles.

6.3 Generalized Krylov–Schur

For Equation (2), the current host-only path builds a B-orthonormal basis,

$$Q^T B Q = I, \tag{17}$$

and applies the self-adjoint operator $B^{-1}A$ in the B inner product. It stores both Q and BQ. Given a residual-like vector f, an expansion step normalizes with

$$\|f\|_B = \sqrt{f^T B f}, \tag{18}$$

computes $y = B^{-1}Aq$, and performs two-pass B-orthogonalization. Restarting rotates the retained basis and recomputes BQ for those columns.

Before iteration, FASTEIG factorizes B once with Eigen SimplicialLDLT or MKL PardisoLDLT. The implementation assumes B is symmetric positive definite. CUDA generalized problems and generalized SAI are not supported.

7 Block Krylov–Schur

Block Krylov–Schur combines a block Lanczos expansion with a compact restart. It is intended for several clustered or multiple eigenpairs when an unrestarted block basis would be too large [4].

The current implementation sets the block width equal to `nev`, rather than reading `blksize`. The requested cycle dimension is `maxdim`, or $\max(2p, 20)$ if `maxdim` is zero. It must be at least $2p$ and is rounded down to a whole number of p-column blocks.

```
Set b = nev and choose cycle dimension k from maxdim
Build a block Krylov cycle with one extra residual block

for restart_cycle = 1, ..., maxiter
    symmetrize and diagonalize the k-by-k projected matrix
    select b wanted Ritz pairs
    form Ritz vectors Y = Q*C
    compute block residual estimates from B_last*C_last

    if converged or at maxiter, return Y

    retain Y as the first block
    retain the residual basis as the second block
    seed the projected matrix with Ritz values and residual coupling
    expand block Lanczos back to the cycle dimension
end
```

The block basis builder uses the shared `BlockKrylovSubspace`, two-pass block Gram–Schmidt, and Cholesky QR. However, the outer cycle, projected matrix, and restart basis are currently host Eigen matrices. A CUDA operator therefore incurs host/device copies at cycle boundaries; this is an executor-aware but not fully device-resident implementation. SAI and generalized problems are not supported.

8 Davidson

Davidson expands a trial subspace with corrections derived from Ritz residuals. Its original strength is that a cheap approximation to $(A-\theta I)^{-1}$, often based on the diagonal of A, can greatly accelerate the lowest eigenpairs [5]. The current FASTEIG implementation has the subspace and locking structure of a block Davidson method, but uses the raw residual as the correction vector; no Davidson preconditioner is active.

```
Orthonormalize the initial nev vectors with Cholesky QR
Perform Rayleigh-Ritz to obtain initial Ritz vectors and residuals
```

```
for outer_cycle = 1, ..., maxiter
    start Q with the current Ritz vectors

    for block = 2, ..., maxblks
        mark vectors satisfying lock_tol as locked
        W = residuals of active Ritz vectors        # no preconditioner today
        orthogonalize W twice against Q
        Q_new = householder_qr(W)
        append Q_new and op(Q_new) to the subspace
        solve the projected eigenproblem
        update Ritz vectors, values, and residuals
    end

    stop if the first nev residuals satisfy tol
end
```

The large trial matrices are host-resident. Operator applications go through the executor-aware `LinearOperator`, so a CUDA operator can be called, but each new host block must cross the memory boundary. In practice this path should be considered CPU/hybrid until the full Davidson state and correction preconditioner are moved into core executor objects.

`maxblks` limits the number of correction blocks per outer cycle and therefore the peak basis size, approximately `maxblks`×`nev`. The method handles the high end by applying the low-end algorithm to $-A$ for `LR`/`LM`.

9 LOBPCG

The Locally Optimal Block Preconditioned Conjugate Gradient method maintains three blocks: current approximations X, previous search directions P, and preconditioned residuals W. A Rayleigh–Ritz minimization in

$$\operatorname{span}\{X, P, W\} \tag{19}$$

chooses locally optimal updates. Its memory is $O(np)$ rather than a growing Krylov basis, and it is especially effective with a good preconditioner [6, 7].

```
Initialize X from v0/random vectors and perform an initial Rayleigh-Ritz step
Initialize P with random unit vectors and compute A*P

for k = 1, ..., maxiter
    lock converged Ritz pairs in order
    W = A*X - X*Lambda for the unconverged columns
    apply preconditioner to W                   # not active in current code
    if required, orthogonalize W against [X, P]
    compute A*W

    perform modified Rayleigh-Ritz in S = [X, P, W]
    if the overlap is poorly conditioned
        orthogonalize W, recompute A*W, and repeat Rayleigh-Ritz
    end

    update X, P, A*X, A*P, and residuals
    stop when all requested residuals satisfy tol
end
```

The modified Rayleigh–Ritz step scales the overlap matrix, attempts a Cholesky reduction of the generalized projected problem, and switches to explicit orthogonalization when conditioning becomes poor. Its SVQB helper repairs or drops weak overlap directions. Locking is ordered so a later Ritz pair is not removed before all preceding requested pairs have converged.

The current preconditioner section is commented out. Consequently W is the unpreconditioned residual, and the implementation should be viewed as LOBPCG without its most important application-specific accelerator. Dense state is host-resident; only operator applications use the selected executor. The high end is handled with the same $-A$ transformation used by Davidson.

10 Chebyshev-filtered subspace iteration

Chebyshev filtering applies a polynomial $p_d(A)$ that amplifies the wanted spectral interval while damping the unwanted one. It then orthonormalizes the filtered block and performs Rayleigh–Ritz. The approach replaces many sequential vector recurrences with block operator applications and is particularly attractive for a sequence of related eigenproblems with a good initial subspace [8].

10.1 Filter construction

The current solver first runs a short, 20-step (or smaller) Lanczos estimate of the spectral lower and upper bounds. Let these estimates be a and b, and let the filter degree be d. With the hard-coded amplification ratio $r = 10^3$, it computes

$$x_\ell = \cosh\left(\frac{\operatorname{acosh}(r)}{d}\right), \tag{20}$$

$$\ell = \frac{2a + bx_\ell - b}{1 + x_\ell}, \tag{21}$$

$$c = \frac{2}{b - \ell}, \qquad e = -\frac{b + \ell}{b - \ell}. \tag{22}$$

The mapped operator is $cA + eI$, and the Chebyshev recurrence is

$$V_0 = V, \qquad V_1 = (cA + eI)V_0, \qquad V_k = 2(cA + eI)V_{k-1} - V_{k-2}. \tag{23}$$

10.2 Implementation-faithful pseudocode

```
Estimate spectral bounds with a short Lanczos run
Create an initial active block plus up to nev spare vectors
Orthonormalize with Cholesky QR

for filter_cycle = 1, ..., maxiter
    apply the degree-d Chebyshev recurrence to active columns
    deflate active columns against locked Ritz vectors
    orthonormalize the current subspace with Cholesky QR
    perform Rayleigh-Ritz
    lock converged requested pairs in order
    activate spare vectors to keep the filtered block populated
    stop when all requested pairs are locked
end
```

The work per cycle is approximately `degree` block applications, plus QR and Rayleigh–Ritz. Too small a degree weakly separates the wanted interval; too large a degree increases work and may magnify scaling errors. The current implementation is host-resident around an executor-aware operator. It handles the high end by filtering $-A$. Generalized problems and SAI are not supported.

11 SPPC

Subspace Projection with Perturbative Corrections is specialized for a hierarchical Hamiltonian whose leading principal submatrix represents a useful smaller configuration space. It begins with eigenvectors of that leading problem, generates Rayleigh–Schrödinger-like correction vectors by solving small shifted systems, and applies Rayleigh–Ritz in the accumulated correction subspace [9].

In the current driver, `maxdim` is interpreted as the split size n_s. The problem constructs

- `H0`, the $n_s \times n_s$ leading principal matrix; and
- `V21`, the lower-left coupling block.

The initial p eigenpairs of `H0` are computed by an internal Krylov–Schur solve and padded with zeros to the full dimension.

```
Solve the leading H0 eigenproblem for p initial eigenpairs

for each requested initial eigenpair (lambda0, v0)
    start correction basis N with v0
    compute the first analytic perturbative correction

    for perturbation order k = 1, ..., maxiter
        form the next correction right-hand side
        solve (H0 - lambda0 I) y = rhs with MINRES
        remove the v0 component and form the full correction vector
        reorthogonalize the correction against N
        append it to N and compute A times the new basis vector
        solve the projected Rayleigh-Ritz problem
        stop when the projected residual satisfies tol
    end
end
```

This is not a generic partitioning strategy: it assumes the leading problem is both much cheaper and physically informative. The present implementation is the independent-eigenpair SPPC variant and is host-oriented. It prints its own per-perturbation progress, and its linear solves use the legacy `operations::minres` callback.

12 RMM-DIIS

Residual Minimization Method with Direct Inversion in the Iterative Subspace is a refinement algorithm. It extrapolates previous corrections to reduce the eigen-residual and combines the current vector with a correction in a small generalized Rayleigh–Ritz problem. It is widely associated with electronic structure calculations [10].

The FASTEIG solver first performs an internal Krylov–Schur solve at tolerance 10^{-3} to obtain initial vectors. Each requested pair is then refined independently:

```
Obtain approximate vectors from a loose-tolerance Krylov-Schur solve

for each approximate eigenvector v
    normalize v; compute theta = v^T A v and residual r
    obtain a correction w1 from the shifted correction callback

    for k = 1, ..., maxiter
        extrapolate from stored correction differences with DIIS least squares
```

```
        recompute theta and r
        obtain a new shifted correction w
        solve the 2-by-2 generalized Ritz problem in span{v, w}
        store the updated vector and operator image
        stop when ||r||/|theta| < tol
    end
end
```

This solver is host-oriented and stores a history whose size grows with `maxiter`. More importantly, the explicit-matrix default `operations::CG` currently solves only a placeholder scaled-identity system. Meaningful RMM-DIIS use therefore requires a suitable user-supplied correction/preconditioning callback or a future replacement of that default. It should be treated as a refinement stage, not as the first method tried from a random vector.

13 Parameter guide

Table 3: Algorithm-facing input options.

Option	Main consumers	Interpretation
`solver`	All	Selects Lanczos, BlockLanczos, BlockKrylovSchur, KrylovSchur, Davidson, LOBPCG, Chebyshev, SPPC, or RMM-DIIS.
`nev`	All	Number of requested pairs. It is also the block width in the current Block Krylov–Schur implementation.
`which`	All	Target-selection code; support differs as described in Section 2.
`tol`	All	Convergence threshold. Intermediate definitions differ; final residuals use Equations (3) and (4).
`maxiter`	All	Scalar expansions for Lanczos; block expansions for Block Lanczos; restart/filter/outer cycles for several other solvers; perturbation/refinement count for SPPC and RMM-DIIS.
`blksize`	Block Lanczos	Number of vectors per block. It is not currently the Block Krylov–Schur width.
`maxdim`	Block Krylov–Schur, SPPC	Restart-cycle dimension for Block Krylov–Schur; leading split size n_s for SPPC. Standard Krylov–Schur currently chooses its own dimension.
`maxblks`	Davidson API	Maximum correction blocks per Davidson outer cycle.
`degree`	Chebyshev	Degree of the polynomial filter.
`shift`	SAI	Target σ in $(A - \sigma I)^{-1}$.
`seed`	Initial-vector paths	Reproducible initial vectors. Executor creation uses the same seeded host sequence before a one-time CUDA copy.
`print_every`	Iterative progress	Progress cadence; in Lanczos and Krylov–Schur it also determines snapshot/check cadence when positive.

The meaning of an "iteration" is solver-specific. Comparing iteration counts across scalar, block, restarted, and polynomial methods is therefore misleading. Matrix/block-operator applications and total runtime are better cross-method measures.

14 Numerical and performance guidance

14.1 Accuracy checks

Always judge final accuracy using the explicitly recomputed residuals. A tiny projected residual with a large final residual indicates a loss of orthogonality, an inconsistent operator, a failed transformation, or a bug in the recurrence/projection coupling. For clustered or repeated eigenvalues, also inspect subspace agreement rather than demanding identical individual eigenvectors: any orthonormal rotation within a degenerate invariant subspace is valid.

CPU and CUDA runs need not stop on exactly the same snapshot. Floating-point reductions occur in different orders, so projected values and residuals can differ at roundoff level. A fixed `seed`, matching options, and final residual comparisons provide the meaningful reproducibility test.

14.2 Memory and restart tradeoffs

Lanczos and Block Lanczos are unrestarted and store every basis vector. This is often acceptable for a few hundred iterations, but memory grows linearly with the iteration count and orthogonalization work grows quadratically. Krylov–Schur and Block Krylov–Schur cap the basis but may require additional cycles after discarding unwanted directions. LOBPCG keeps only a few p-column blocks and therefore has the smallest predictable large-vector footprint among the general block methods.

14.3 CPU and GPU expectations

Moving data to a GPU is useful only when enough work is done before it returns. Lanczos keeps the basis, sparse products, reorthogonalization, projected workspace, and final Ritz multiplication behind CUDA executor objects. Block Lanczos increases arithmetic intensity through block kernels. In contrast, the current hybrid solvers repeatedly materialize Eigen host matrices; a CUDA operator alone does not make those algorithms GPU-efficient.

CUDA startup, matrix upload, and SAI factorization are one-time costs. Include them for end-to-end timing, but separate them when studying repeated solves that can reuse an executor or factorization. For SAI, compare the cost of the factorization plus triangular solves against the reduction in Krylov iterations.

15 Implementation map

The main source locations are listed below so that the mathematical discussion can be connected to the code without placing code-level details in the user's guide.

Location	Responsibility
`src/*Solver.cpp`	Solver-level loops, options, stopping, locking, and restart policies.
`include/core/Executor.hpp`	Memory-space and backend factory interface.
`core/Vector`, `core/MultiVector`	Executor-owned vector and block-vector primitives.
`core/LinearOperator`, `core/CsrMatrix`	Abstract operator and explicit CSR implementations.
`core/KrylovSubspace`	Shared scalar and block basis-building steps.
`core/Orthogonalization`	CPU/CUDA QR, SVQB, and block Gram–Schmidt.
`core/LanczosWorkspace`	CPU/CUDA projected-matrix recording, snapshots, restarts, and finalization.
`core/RitzExtraction`	Ritz ordering, residual estimates, SAI back-transformation, and progress snapshots.
`src/gpu/Cuda*.cpp`	CUDA vectors, multivectors, CSR, workspace, and cuDSS shift-invert operator.
`src/fasteig_problem.cpp`	Executor/operator construction, SAI and B factorization, solver dispatch preparation.

16 References

References

[1] C. Lanczos, "An iteration method for the solution of the eigenvalue problem of linear differential and integral operators," *Journal of Research of the National Bureau of Standards*, 45, 255–282, 1950. doi:10.6028/jres.045.026.

[2] G. H. Golub and R. R. Underwood, "The Block Lanczos Method for Computing Eigenvalues," in *Mathematical Software III*, 361–377, 1977. doi:10.1016/B978-0-12-587260-7.50018-2.

[3] G. W. Stewart, "A Krylov–Schur Algorithm for Large Eigenproblems," *SIAM Journal on Matrix Analysis and Applications*, 23(3), 601–614, 2001. doi:10.1137/S0895479800371529.

[4] Y. Zhou and Y. Saad, "Block Krylov–Schur method for large symmetric eigenvalue problems," *Numerical Algorithms*, 47, 341–359, 2008. doi:10.1007/s11075-008-9192-9.

[5] E. R. Davidson, "The iterative calculation of a few of the lowest eigenvalues and corresponding eigenvectors of large real-symmetric matrices," *Journal of Computational Physics*, 17(1), 87–94, 1975. doi:10.1016/0021-9991(75)90065-0.

[6] A. V. Knyazev, "Toward the Optimal Preconditioned Eigensolver: Locally Optimal Block Preconditioned Conjugate Gradient Method," *SIAM Journal on Scientific Computing*, 23(2), 517–541, 2001. doi:10.1137/S1064827500366124.

[7] J. A. Duersch, M. Shao, C. Yang, and M. Gu, "A Robust and Efficient Implementation of LOBPCG," *SIAM Journal on Scientific Computing*, 40(5), C655–C676, 2018. doi:10.1137/17M1129830.

[8] Y. Zhou, Y. Saad, M. L. Tiago, and J. R. Chelikowsky, "Self-consistent-field calculations using Chebyshev-filtered subspace iteration," *Journal of Computational Physics*, 219(1), 172–184, 2006. doi:10.1016/j.jcp.2006.03.017.

[9] D. M. Roh, D. Lee, P. Maris, E. Ng, J. P. Vary, and C. Yang, "Accelerating eigenvalue computation for nuclear structure calculations via perturbative corrections," *Journal of Computational Physics*, 531, 113921, 2025. doi:10.1016/j.jcp.2025.113921.

[10] G. Kresse and J. Furthmüller, "Efficient iterative schemes for ab initio total-energy calculations using a plane-wave basis set," *Physical Review B*, 54, 11169–11186, 1996. doi:10.1103/PhysRevB.54.11169.

[11] Y. Saad, *Numerical Methods for Large Eigenvalue Problems*, revised edition, SIAM, 2011. doi:10.1137/1.9781611970739.

[12] T. Fukaya, R. Kannan, Y. Nakatsukasa, Y. Yamamoto, and Y. Yanagisawa, "Shifted Cholesky QR for Computing the QR Factorization of Ill-Conditioned Matrices," *SIAM Journal on Scientific Computing*, 42(1), A477–A503, 2020. doi:10.1137/18M1218212.

[13] H. Anzt et al., "Ginkgo: A Modern Linear Operator Algebra Framework for High Performance Computing," *Journal of Open Source Software*, 5(52), 2260, 2020. doi:10.21105/joss.02260.